

Controller Prompting System for Steam Deck

Executive Summary

We propose a system that lets Steam Deck users select and compose complex “elite” prompts via the gamepad, with intelligent autocomplete and full use of the Deck’s unique inputs (including the four rear buttons L4/L5/R4/R5). The design spans hardware specs (buttons/axes), SteamOS and Steam Input APIs, mapping formats, and emulation layers. We survey existing autocomplete and prompt-pattern libraries, then outline a data schema linking prompt tokens to controller inputs, with priority/weight and context metadata. Autocomplete algorithms (e.g. prefix trees, fuzzy search) and structured prompt templates will be used to deliver fast suggestions with minimal latency. We consider integration options (native Steam Input configs vs middleware/daemon/overlay) and compare them in a summary table. Finally, we cover testing, security/privacy, and accessibility. Throughout, official Valve/Steam documentation is cited when available ¹ ² .

Steam Deck Hardware & Input Constraints

The Steam Deck packs standard console controls plus extra features ³ . It has A/B/X/Y buttons, a D-pad, two analog sticks, L1/R1 bumpers, L2/R2 analog triggers, View/Menu buttons, two large trackpads (each with touch and click), a gyro, and **four assignable rear grip buttons (L4, L5 on the left; R4, R5 on the right)** ³ . All these inputs can be mapped via Steam Input. In hardware terms, L4/L5 and R4/R5 are digital push-buttons (unlike analog triggers), giving four extra generic button events.

One caveat: in Steam’s default desktop/gamepad mode, the Deck appears as an XInput (Xbox 360) controller to games ⁴ . That virtual controller actually hides the four rear grips. However, the Linux **hid-steam** driver (in desktop mode or custom apps) exposes a “Steam Deck” device with all buttons, including L4/L5/R4/R5, at the kernel level ⁵ . In practice, games running under Steam (or Proton) will rely on Steam Input to surface those extra buttons. Thus any prompt system must work with Steam Input mappings, not raw kernel events.

SteamOS Input APIs and Steam Input System

Valve’s Steam Input system is a feature-complete input layer that lets games define abstract “actions” which players map to any controller ⁶ ² . As Valve puts it, Steam Input is a “flexible action-based API” covering Xbox, PlayStation, Nintendo, Steam Controllers, etc ² . Game developers supply an **In-Game Actions (IGA) file** in Valve’s VDF (Valve Data Format) that names action sets (e.g. “Menu”, “Gameplay”) and actions within them ¹ . For example:

```
"actions" {  
  "gameplay" {  
    "title" "#ActionSetGameplay"
```

```

    "ButtonA" { "title" "Jump" }
    "AnalogTrigger" { "title" "Throttle" }
  }
}

```

This file (game_actions_APPID.vdf) defines all game actions. Valve's system then lets players remap any controller input (including Steam Deck's L4-5/R4-5) to those actions. A separate **Action Manifest** (optional) can bundle multiple controller configs and localizations ⁷.

Developers typically integrate via the **ISteamInput API**. Once initialized, the API can query controller handles and action set handles, then read digital or analog action states each frame. For example, `SteamInput()->GetDigitalActionData(controller, jumpHandle)` would tell if the "Jump" action (mapped to, say, A button) is pressed. The API also provides ways to get the *origin* (which physical button) is bound to an action, to show correct on-screen prompts ⁸. In short, Steam Input abstracts the hardware so games only see logical actions ². Our prompt system could either speak this API (if integrated in a game), or more likely live outside the game as a middleware or overlay that listens to controller input and then triggers text injection.

Controller Emulation & Game Compatibility

Steam Input acts as an emulation layer so that even games without built-in Steam Input support still see a gamepad. On Windows, Steam overlay hooks XInput/DirectInput/RawInput APIs and injects a *virtual Xbox controller* for the game ⁹. On Steam Deck's Linux (SteamOS) environment, a driver provides an emulated gamepad to the application. In practice, virtually all Steam Deck games see an Xbox-style pad: A/B/X/Y, bumpers/triggers, D-pad, etc., and for example a PS4 trackpad click or PS options button both end up mapped to "XInput Start" ¹⁰. Extra inputs beyond standard buttons (like the Deck's grips) often have to be "grafted" onto unused Xbox buttons or accessed via Steam Input configs. As [35] notes, some extra controls may be duplicated to existing inputs.

Over 2000 Steam games are known to use Steam Input (at least for one controller type) ¹¹. In Proton (for Windows games), Steam's input emulation likewise appears as an XInput device. This means compatibility is generally broad: you can remap keys or grips to keyboard strokes, mouse movement, or Xbox buttons on-the-fly via Steam's configurator. For our prompt system, two points stand out: 1. **Mapping to Text:** Many games allow key bindings (e.g. chat key). We can map L4/L5 combos to simulate keyboard typing (via Steam config) if needed.

2. **Graphical Overlay:** Most games will block typical UI overlays, so a Steam Input overlay or cheat menu is risky. Better to rely on Steam's own overlay (triggered by a hotkey) or a safe channel (e.g. a background daemon reading inputs and injecting through X11/Wayland).

In summary, any solution must work within Steam's controller abstraction layer, emulating keystrokes or injecting chat text. Using Steam Input's action sets and layers is safe for games, whereas poking at raw input might break compatibility.

Autocomplete Libraries, Tokenization & UI Patterns

- **Autocomplete Libraries:** Traditional autocomplete often uses prefix trees (tries), sorted lists, or fuzzy-search algorithms. For example, the open-source [autoComplete.js](#) is a tiny JS lib for prefix matches, while larger systems (in search engines) use indexed keywords. Since our prompt library could be very large, we might lean on a compact data structure (e.g. a compressed trie or T9-like scheme) for prefix matching. There aren't off-the-shelf "controller autocomplete" libraries, but mobile/cellphone input (T9, Swype) offer analogies: they convert limited keys to many letters. We might also consider using a tiny on-device LLM or n-gram model for true predictive suggestions, but at least a basic string autocomplete (filtering a list of known prompts) is a good start.
- **Prompt Libraries & Tokenization:** In the AI community, huge prompt collections exist (e.g. FlowGPT) where users share curated ChatGPT prompts, often tagged by domain. These prompts can be tokenized by word or by subword (Byte Pair Encoding) for efficient storage and search ¹². We can adopt a simple scheme: split prompts into whitespace-separated tokens (keywords) and index those. Optionally, apply a subword/BPE tokenization like HuggingFace's GPT models do, allowing partial matches mid-word ¹². For example, "weather forecast tomorrow" might index ["weather", "forecast", "tomorrow"] plus maybe sub-tokens for fuzzy typing. The more advanced approach is storing full tokens and prefixes (search trees) for quick suggestion.
- **UI/UX Patterns:** Typing with a gamepad sucks without help. Historically, consoles have tried many methods ¹³ ¹⁴. Older systems used on-screen keyboards (hunt-and-peck QWERTY or ABC grids). The PSP famously had a letter-group + autocomplete UI: you navigate letters in categories, then get word suggestions ¹³. Steam Big Picture (and the Deck's native keyboard) use an *alphabetical hierarchical* layout: push a direction with the stick to group letters, press a face button to drill down, etc ¹⁴. That system is fast once learned, but new users might struggle. We can borrow ideas:
 - **Radial/Circular Menus:** With the Deck's touchpads or sticks, one could implement a radial suggestion menu (360° input + face button for choice).
 - **Dual-Analog Entry:** The GameDev article even experimented with dual-stick cursor to speed up typing ¹⁵. However, such exotic methods have a learning curve.
 - **Autocomplete Facelift:** A more modern take is to overlay a list of suggestions next to your cursor or selection zone. The user can scroll through them via bumpers or stick. This mimics smartphone autocomplete but with gamepad navigation.
 - **Template Fill-in:** We should also support selecting from structured templates. For example, a prompt template could be "Write a [Genre] story about [Character]" with blanks navigable by D-pad. This reduces pure typing.In all cases, the UI must be legible on TV/handheld and use big, clear text/buttons, since controlling via thumbstick is imprecise.

Prompt Library Data Schema

We propose a JSON-like schema for storing prompt entries with metadata. Each prompt entry might look like:

```

{
  "id": "p123",
  "text": "Attack the nearest enemy with full force",
  "tokens": ["attack","nearest","enemy","full","force"],
  "category": "combat",
  "weight": 100,
  "lastUsed": 1680000000,
  "contexts": ["battle","objective"],
  "bindings": [
    {"input": "L4", "action": "selectCategory:combat"},
    {"input": "R4", "action": "selectAction:attack"},
    {"input": "A", "action": "confirm"}
  ]
}

```

- **id/text**: the prompt ID and full string.
- **tokens**: an array of key terms (space- or punctuation-separated) for indexing and autocomplete matching.
- **category/contexts**: metadata to filter suggestions (e.g. “this is a combat prompt”, or tags for situational context).
- **weight/priority**: a numeric score indicating frequency or importance (higher = shown first). We could update `lastUsed` and bump `weight` on selection to learn from the user.
- **bindings**: optional default button sequences or gesture hints that yield this prompt; these show up in a help list. For example, binding `L4 + R4` might be documented to trigger “Attack” prompts.
- **conflictTags**: (not shown above) could list other prompts that shouldn’t be suggested together (e.g. mutually exclusive suggestions).

This schema lets the system quickly find candidate prompts containing a given token prefix, rank them by weight and recency, and ensure contextual relevance. For conflict resolution, if two high-priority prompts conflict (e.g. both bind to L4-L5), the system might demote one or require double-confirmation.

Autocomplete Algorithms & Structured Templates

For autocomplete, a **prefix-search** algorithm is simplest: maintain a trie or sorted list of all tokens. As the user types (via D-pad selecting letters or one of a few “letters” offered), filter the prompt list by token prefix. A more advanced approach is **fuzzy matching** (allow one or two typos); libraries like [autoComplete.js](#) use bitap algorithms, but they are JS/browser. On Deck, a custom C++/Rust prefix tree can do millions of ops very fast. Latency should stay below ~50ms per keypress to feel instant. To reduce work, one can precompute indexes: e.g., a map from first-letter to candidate list, then filter smaller lists on each additional letter.

Structured templates could use similar matching. For example, have a list of templates like “Describe a map of [location] with weather [conditions].” In the UI, when the user selects the “Map” category (with a token “map”), show these templates. A template might have placeholders (e.g. [location]) that the user can cycle through predefined options using the d-pad.

Performance Considerations: With thousands of prompts/tokens, search must be efficient. A trie of all token prefixes is fast and memory-light; a database (e.g. SQLite FTS) could be used but might be overkill. Caching recent queries and limiting suggestions (e.g. top-10 results) prevents lag. We should also test on Deck's hardware (similar to a mid-range laptop CPU) to ensure acceptable response time.

Integration Approaches

Several integration strategies are possible:

- **Native Steam Input Config:** Use Steam's own controller configuration system to map buttons to macros or text. For example, Valve's "Shortcuts" feature lets a button open a keyboard or custom text. We could pack our prompt library into a custom Steam config (controller_config files) so pressing L4 might open a radial menu of prompts via Steam's "On-Screen Keyboard" or text macro tool. *Pros:* Uses built-in system, no extra software. *Cons:* Steam's config UI is not designed for huge autocomplete lists; also Steam's VDF is static (difficult to update dynamically).
- **Middleware/Daemon:** A background program running on SteamOS listens for a special toggle (e.g. L4+L5 together) and then overlays its own UI on top of any game. It intercepts controller events (via evdev or SDL) while in "prompt mode," shows suggestions (perhaps using a minimal graphics overlay or a transparent browser widget), and on selection it injects keystrokes (X11/Wayland) or uses Steam's API to send text. *Pros:* Very flexible, can implement rich UI and algorithms. *Cons:* Complex; might conflict with game input or require root access or udev rules. Would not easily work in Proton/Windows games without hooking into Wine/Proton internals.
- **Steam Overlay Plugin:** In theory one could write a Steamworks plugin or use Steam's Workshop to supply an in-game overlay (similar to performance overlay) that appears when a hotkey is pressed. However, Valve doesn't officially support third-party overlays on Deck aside from Steam's own HUD. Realistically, this is not an option for non-validated devs.
- **External Controller Device:** Use an external gadget (like a Bluetooth keyboard or Raspberry Pi) that the Deck sees as another input device. That gadget could run the prompt UI and send resulting keystrokes. *Pros:* No change to SteamOS. *Cons:* Requires extra hardware and pairing, messy.
- **Proton Prefix Injection:** For Windows games, one could hack at the Proton layer (Wine) to detect specific controller codes and paste text. This is highly game-specific and brittle.

The most viable approaches are **Steam Input Config** and **Middleware/Daemon**. Below is a quick comparison:

Integration Approach	Mechanism	Pros	Cons
Steam Input Config	Custom Steam controller_config VDF mappings (macros or shortcuts)	Built into Steam; works in any game on Deck ¹ . No extra software.	Limited by Steam's preset UI (difficult to show long lists). Static configs need manual update.

Integration Approach	Mechanism	Pros	Cons
Middleware/Daemon	Userland app intercepts input, draws custom UI, injects key events	Full control over UI/logic; can dynamically update prompt library and context.	Complex to implement; potential conflicts. Not natively supported by Steam; may require workaround to inject into games.
Steam Overlay (unsupported)	A plugin or HUD overlay within SteamOS (theoretical)	Seamless in Steam ecosystem (if it worked)	Valve does not support third-party overlays. Hard to implement.
External Device	Another input device acts as “controller”; translates to keystrokes	Offloads UI entirely; Steam treats it as separate controller.	Requires extra hardware; pairing issues; less elegant.

Testing Plan

A rigorous test plan is critical. Key steps include:

- **Unit/Integration Tests:** Validate the autocomplete engine on large prompt sets. Test token matching, fuzzy ranking, and edge cases (empty input, common prefixes). Use automated tests to feed synthetic input sequences and verify correct suggestions.
- **Hardware-in-the-Loop:** On an actual Steam Deck (and in SteamOS emulators), test that each button/event is recognized and mapped correctly. In particular, confirm that L4/L5/R4/R5 events are received by the system and by Steam Input remapping.
- **Configuration Tests:** If using Steam Input configs, test loading custom controller_config files (in `~/.steam/steam/controller_config/`) and ensure Steam picks them up. Use tools like `steamcmd` to simulate local users saving configs (see [15†L395-L404]).
- **Game Compatibility Tests:** Run a suite of games (both native Linux and Proton Windows) to ensure the prompt UI can be triggered and text injected without breaking the game. For proton titles, ensure no deadlocks or crashes. Validate on a range of genres (FPS, RPG, MMO chat, etc.).
- **Performance Benchmarking:** Measure input-to-suggestion latency. Use a deck or similar hardware to simulate worst-case library size; ensure UI updates in <100ms. Also check memory usage (autocomplete indexes should fit comfortably in Deck’s RAM).
- **User Testing:** Since UI is novel, perform UX testing. Does a typical gamer find the UI usable? Are key sequences intuitive? Check on handheld mode and docked/TV mode. Evaluate accessibility (font sizes, focus indicators). Incorporate feedback to refine navigation (e.g. maybe add shortcuts or voice hints).

Automation tools might include Steamworks SDK’s debug logging or custom printf to track action triggers. For security, fuzz test with malicious prompt strings. Finally, document all tests for Steam Deck Verified compliance; Valve requires clear evidence of stability and safety.

Security, Privacy & Accessibility

Privacy & Security: The prompt library may contain user-provided or downloaded text. We should treat it as possibly sensitive (in case prompts encode personal info). All processing can be kept on-device; avoid uploading user prompts to any server. If using any third-party autocomplete model (LLM), ensure it runs locally or anonymized. The system should not elevate privileges or expose the Deck to remote code. Note: remapping inputs or injecting keys is benign from a game-viewpoint but could be abused (e.g. auto-cheat macros). We assume trusted use (user manually enabling it). Still, the software should clearly inform users what remappings it does.

From a compatibility standpoint, our solution should not tamper with game executables, to avoid violating EULAs. Using Steam Input or standard input injection (e.g. X11 key events) is generally safe. Avoid game memory hacks or writing new DLLs into Proton games.

Accessibility: Controller-based text input is inherently harder than keyboard. We must support different needs:

- **Visual:** Ensure high-contrast UI and legible font (Deck screen is smallish). Provide voice-over text for suggestions if SteamOS has TTS.
- **Motor:** Offer multiple navigation options: D-pad, stick scrolling, and maybe gyro-motion cursor (Deck gyro) to point at letters.
- **Auditory/Cognitive:** Reading long lists can be annoying. We should minimize steps with good autocompletion (fewer keystrokes). Offer audio confirmation (haptic or sound) on selection.
- **Localization:** If the user's system language is non-English, prompt texts and UI should adapt. Steam's localization system could be leveraged (the IGA file supports localized action titles ¹).
- **Colorblindness:** Avoid color-only cues; use icons or shapes too.
- **Customizability:** Let users adjust navigation (e.g. swap L4/L5 roles) and text speed.
Given the target is gamers on a handheld, these should suffice.

Example Prompt Mapping Table

Below is a sample mapping of some fictional "elite" prompts to Steam Deck buttons. Each prompt is tokenized and mapped to controls including L4/L5 and R4/R5:

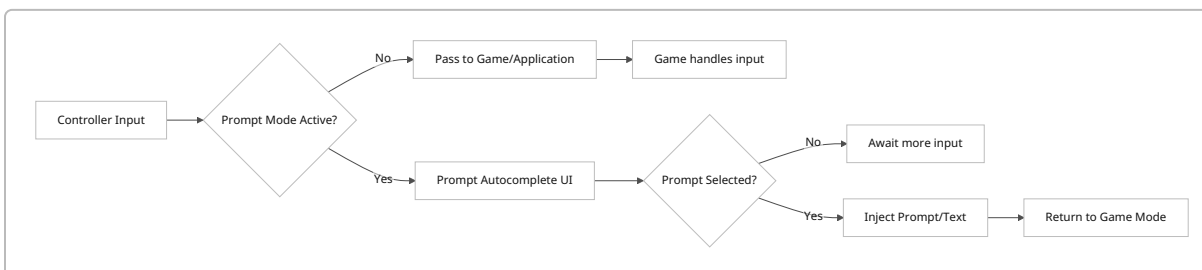
Prompt Text	Tokens	Example Control Mapping
"Power Attack Sequence"	<i>power, attack, sequence</i>	L4 → "Combat" category → R4 = Select "Power Attack" → A = Execute
"Elite Spell: Fireball"	<i>elite, spell, fireball</i>	L5 = Quick Spell Menu; scroll to "Fireball" with stick; R5 = Confirm
"Stealth Recon Maneuver"	<i>stealth, recon, maneuver</i>	L4+L5 (dual-press) to open "Tactical" menu; R4 = "Recon," R5 = "Cancel," A = "Execute Maneuver"
"Summon Drone Squad"	<i>summon, drone, squad</i>	R4 = open "Summons," L4 = scroll to "Drone Squad," A = cast

Prompt Text	Tokens	Example Control Mapping
"Emergency Heal Pulse"	<i>emergency, heal, pulse</i>	L5 = Special Actions; R4 = "Heal Pulse," then trigger effect with B

Each entry shows how tokens are identified (left column) and one possible button sequence (right). In a real UI, the user might see "power attack sequence" suggestions after typing "pow", or find it via menu paths. The key point is that **every prompt is reachable by some combination of Deck buttons (including L4/L5, R4/R5)**. The above is illustrative; a real implementation would fine-tune these mappings.

Input Flow (Mermaid Diagram)

Below is a flowchart of the input-processing architecture. It shows how controller events enter the system and either pass through to the game or are handled by the prompt UI:



In words: all button events first hit a mode check. If the user has not activated the prompt system (perhaps via a special combo), input goes to the running game normally. When prompt mode is on, input navigates the autocomplete UI: typing letters or selecting categories yields suggestion lists, and once a prompt is chosen, the system injects it (e.g. by simulating keyboard input or triggering a game action). Then control returns to the game.

Integration Options Comparison

Approach	Description	Pros	Cons
Steam Config Only	Leverage Steam Input controller configs (VDF). Map buttons or macros to preloaded prompt scripts or keyboard shortcuts via Steam's UI ¹ .	No extra software; works on any Steam Deck game with config.	Very limited UI; Steam's config interface isn't made for browsing thousands of prompts. Static – changes require re-uploading config.
Background Daemon	A custom service on SteamOS captures inputs (e.g. via evdev/SDL), displays a UI overlay (via graphics library), and injects output (key events) to the OS.	Full control, dynamic updates, rich UI possible. Independent of Steam's tooling.	Complex to build. Input conflict risk. May need elevated permissions or careful sandboxing.

Approach	Description	Pros	Cons
Proton/Wine Hook	For Windows games, modify Proton/Wine input handlers to intercept a special key and launch the prompt UI.	Integrated into Windows layer; no extra device needed for those games.	Extremely high effort per-game. Likely breaks with updates; not universal.
External Controller	Use an extra Bluetooth device (e.g. programmable keypad) to run the prompt UI. Deck sees output as keyboard or controller.	No software changes on Deck.	Requires additional hardware; less elegant.

Each approach trades ease-of-implementation against user experience flexibility. A Steam Config-only solution is simplest (and is fully supported by Valve), but suffers from UI limits. A middleware daemon offers the smoothest experience but is an independent hack. Depending on resources, a mixed strategy could work: e.g. use Steam Config for a basic “open prompt menu” binding, then have a small background app pop up the UI.

Conclusion

Building an “elite prompt” system on Steam Deck demands combining knowledge of the Deck’s hardware, Valve’s Steam Input system, and text UI design. We must respect Steam’s controller abstraction (using action mappings and the Deck’s extra buttons) while providing a fast, intuitive autocomplete-driven interface for composing prompts. Our plan covers all dimensions: from hardware specs and input APIs (citing Valve docs ¹ ²) to UI patterns for gamepad text entry ¹³ ¹⁴ . With a clear data schema, efficient search algorithms, and thorough testing, this system can empower Deck users to leverage a vast prompt library seamlessly.

Sources: Valve/Steamworks official docs ¹ ² and related analyses, plus gaming UI research ¹³ ¹⁴ ⁹ ⁴ , informed the above design.

¹ ⁶ In-Game Actions File (Steamworks Documentation)
https://partner.steamgames.com/doc/features/steam_controller/iga_file

² ⁸ ISteamInput Interface (Steamworks Documentation)
<https://partner.steamgames.com/doc/api/isteaminput>

³ Steam Deck :: Tech Specs
<https://www.steamdeck.com/en/tech>

⁴ ⁵ Steam Deck, HID, and libmanette adventures – Just another blog
<https://blogs.gnome.org/alicem/2024/10/24/steam-deck-hid-and-libmanette-adventures/>

⁷ Action Manifest Files (Steamworks Documentation)
https://partner.steamgames.com/doc/features/steam_controller/action_manifest_file

⁹ ¹⁰ ¹¹ Steam Input Gamepad Emulation - Best Practices (Steamworks Documentation)
https://partner.steamgames.com/doc/features/steam_controller/steam_input_gamepad_emulation_bestpractices

12 Byte-Pair Encoding tokenization - Hugging Face

<https://huggingface.co/learn/llm-course/en/chapter6/5>

13 14 15 Towards Better Gamepad Text Input

<https://www.gamedeveloper.com/design/towards-better-gamepad-text-input>